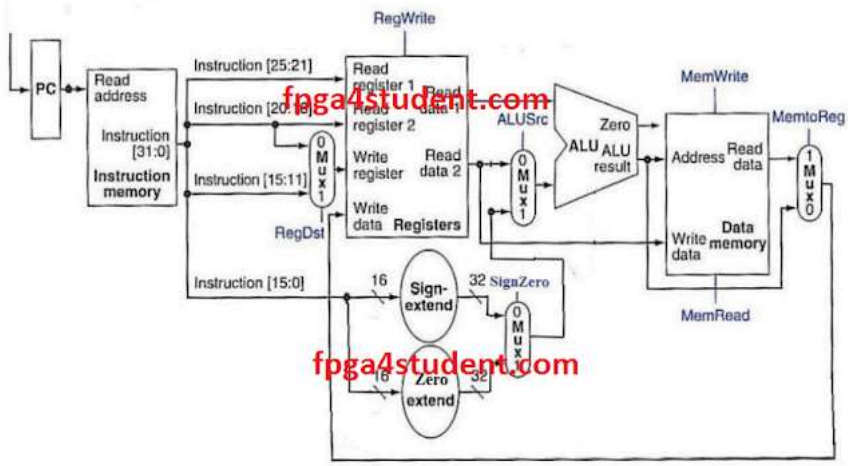
Basic digital logic components in Verilog HDL

In this Verilog project, basic blocks in digital logic design such as D-Flip-Flop, adders, ALU, registers, memory, multiplexers, decoders, counters, etc. are implemented in Verilog HDL for beginners.



Verilog code for full adder:

```
//fpga4student.com: FPga projects, Verilog projects, VHDL projects
// Verilog code for full adder
 module adder(sum,cout,a,b,cin);
 input  a,b,cin;
 output cout,sum;
 // sum = a xor b xor cin
 xor #(50) (sum,a,b,cin);
 // carry out = a.b + cin.(a+b)
 and #(50) and1(c1,a,b);
 or #(50) or1(c2,a,b);
 and #(50) and2(c3,c2,cin);
 or #(50) or2(cout,c1,c3);
 endmodule
```

Verilog code for D Flip Flop:

```
//fpga4student.com: FPga projects, Verilog projects, VHDL projects
// Verilog code for Flip Flop
module D_FF (q, d, rst_n, clk,init_value);
 output q;  //fpga4student.com: FPga projects, Verilog projects, VHDL projects
 input d, rst_n, clk,init_value;
 reg q; // Indicate that q is stateholding
 always @(posedge clk or negedge rst_n)
 if (~rst_n)
 q <= init_value;     // On reset, set to 0
 else
 q <= d; // Otherwise out = d

 endmodule
```

Verilog code for 4-bit counter:

```
 //fpga4student.com: FPga projects, Verilog projects, VHDL projects
// Verilog code for counters
module counter(count,enable,clk,rst_n);
  input enable,clk,rst_n;
  output reg[3:0] count;
  always @(posedge clk or negedge rst_n)
  begin
   if(~rst_n) counter <= 4'b0000;
```

```
   else if(enable)
    counter <= counter + 4'b0001;
  end  //fpga4student.com: FPga projects, Verilog projects, VHDL projects
 endmodule
```

Verilog code for Linear Feedback Shift Register:

```
 //fpga4student.com: FPga projects, Verilog projects, VHDL projects
// Verilog code for Shift Register
module internal_xor_5_bit_lfsr(input clk, rst_n,
                 input[4:0] S_initial,
                 output[4:0] Sout);
 wire [4:0] s_reg;
 wire d_xor;
 D_FF s0(.q(s_reg[0]), .d(s_reg[4]), .rst_n(rst_n), .clk(clk),.init_value(S_initia
 D_FF s1(.q(s_reg[1]), .d(s_reg[0]), .rst_n(rst_n), .clk(clk),.init_value(S_initia
 xor xor_u(d_xor,s_reg[1],s_reg[4]);
 D_FF s2(.q(s_reg[2]), .d(d_xor), .rst_n(rst_n), .clk(clk),.init_value(S_initial[2
 D_FF s3(.q(s_reg[3]), .d(s_reg[2]), .rst_n(rst_n), .clk(clk),.init_value(S_initia
 D_FF s4(.q(s_reg[4]), .d(s_reg[3]), .rst_n(rst_n), .clk(clk),.init_value(S_initia
 assign Sout = s_reg;
 endmodule
```

Verilog code for ALU:

```
//fpga4student.com: FPga projects, Verilog projects, VHDL projects
// Verilog code for ALU
module alu(
      input          [15:0]     a,          //src1
      input          [15:0]     b,          //src2
      input          [2:0]     alu_control,     //function sel
      output     reg     [15:0]     result,          //result
      output zero
   );
 always @(*)  //fpga4student.com: FPga projects, Verilog projects, VHDL projects
 begin
      case(alu_control)
      3'b000: result = a + b; // add
      3'b001: result = a - b; // sub
      3'b010: result = a & b; // and
      3'b011: result = a | b; // or
      3'b100: begin if (a<b) result = 16'd1;
                     else result = 16'd0;
                     end
      default:result = a + b; // add
      endcase
 end
 assign zero = (result==16'd0) ? 1'b1: 1'b0;
 endmodule
```

Verilog code for multiplexer:

```
//fpga4student.com: FPga projects, Verilog projects, VHDL projects
// Verilog code for multiplexer
 module mux2x5to5( AddrOut,Addr0, Addr1, Select);
 output [4:0] AddrOut; // Address Out
 input [4:0] Addr0, Addr1; // Address In 1 and 2
 input Select;
 mux21 mux0(AddrOut[0],Addr0[0],Addr1[0],Select);
 mux21 mux1(AddrOut[1],Addr0[1],Addr1[1],Select);
 mux21 mux2(AddrOut[2],Addr0[2],Addr1[2],Select);
 mux21 mux3(AddrOut[3],Addr0[3],Addr1[3],Select);
 mux21 mux4(AddrOut[4],Addr0[4],Addr1[4],Select);
 endmodule
 `timescale 1 ps / 100 fs
 module mux2_1(O,A,B,sel);
```

```
// sel = 0 => O = A
// sel = 1 => O =B
output O;
input A,B,sel;
not #(50) not1(nsel,sel);
and #(50) and1(O1,A,nsel);
and #(50) and2(O2,B,sel);
or #(50) or2(O,O1,O2);
endmodule
```

Verilog code for register file:

```
//fpga4student.com: FPga projects, Verilog projects, VHDL projects
// Verilog code for register file
 module register_file
 (
      input                    clk,
      input                    rst,
      // write port
      input                    reg_write_en,
      input          [2:0]     reg_write_dest,
      input          [15:0]     reg_write_data,
      //read port 1
      input          [2:0]     reg_read_addr_1,
      output          [15:0]     reg_read_data_1,
      //read port 2
      input          [2:0]     reg_read_addr_2,
      output          [15:0]     reg_read_data_2
 );
      reg     [15:0]     reg_array [7:0];
      // write port
      //reg [2:0] i;
      always @ (posedge clk or posedge rst) begin
           if(rst) begin
                reg_array[0] <= 16'b0;
                reg_array[1] <= 16'b0;
                reg_array[2] <= 16'b0;
                reg_array[3] <= 16'b0;
                reg_array[4] <= 16'b0;
                reg_array[5] <= 16'b0;
                reg_array[6] <= 16'b0;
                reg_array[7] <= 16'b0;
           end
           else begin
                if(reg_write_en) begin
                     reg_array[reg_write_dest] <= reg_write_data;
                end
           end
      end
      assign reg_read_data_1 = ( reg_read_addr_1 == 0)? 16'b0 : reg_array[reg_read
      assign reg_read_data_2 = ( reg_read_addr_2 == 0)? 16'b0 : reg_array[reg_read
 endmodule
```

Verilog code for instruction memory:

```
//fpga4student.com: FPga projects, Verilog projects, VHDL projects
// Verilog code for instruction memory
 module instr_mem          // a synthesisable rom implementation
 (
      input     [15:0]     pc,
      output wire     [15:0]          instruction
 );
      wire [3 : 0] rom_addr = pc[4 : 1];
      /* lw     $3, 0($0) --
           Loop:     slti $1, $3, 50
```

```
            beq $1, $0, Skip
            add $4, $4, $3
            addi $3, $3, 1
            beq $0, $0, Loop--
            Skip
 */
      reg [15:0] rom[15:0];
      initial
      begin
                rom[0] = 16'b1000000110000000;
                rom[1] = 16'b0010110010110010;
                rom[2] = 16'b1101110001100111;
                rom[3] = 16'b1101110111011001;
                rom[4] = 16'b1111110110110001;
                rom[5] = 16'b1100000001111011;
                rom[6] = 16'b0000000000000000;
                rom[7] = 16'b0000000000000000;
                rom[8] = 16'b0000000000000000;
                rom[9] = 16'b0000000000000000;
                rom[10] = 16'b0000000000000000;
                rom[11] = 16'b0000000000000000;
                rom[12] = 16'b0000000000000000;
                rom[13] = 16'b0000000000000000;
                rom[14] = 16'b0000000000000000;
                rom[15] = 16'b0000000000000000;
      end
      assign instruction = (pc[15:0] < 32 )? rom[rom_addr[3:0]]: 16'd0;
 endmodule
```

Verilog code for decoder:

```
//fpga4student.com: FPga projects, Verilog projects, VHDL projects
// Verilog code for decoder
module decoder(WriteEn,RegWrite, WriteRegister);
 input RegWrite;
 input [4:0] WriteRegister;
 output [31:0] WriteEn;
 wire [31:0] OE; // Output Enable
 dec5to32 dec(OE,WriteRegister);
 assign WriteEn[0]=0;
  and #(50) gate1(WriteEn[1],OE[1],RegWrite);
  and #(50) gate2(WriteEn[2],OE[2],RegWrite);
  and #(50) gate3(WriteEn[3],OE[3],RegWrite);
  and #(50) gate4(WriteEn[4],OE[4],RegWrite);
  and #(50) gate5(WriteEn[5],OE[5],RegWrite);
  and #(50) gate6(WriteEn[6],OE[6],RegWrite);
  and #(50) gate7(WriteEn[7],OE[7],RegWrite);
  and #(50) gate8(WriteEn[8],OE[8],RegWrite);
  and #(50) gate9(WriteEn[9],OE[9],RegWrite);
  and #(50) gate10(WriteEn[10],OE[10],RegWrite);
  and #(50) gate11(WriteEn[11],OE[11],RegWrite);
  and #(50) gate12(WriteEn[12],OE[12],RegWrite);
  and #(50) gate13(WriteEn[13],OE[13],RegWrite);
  and #(50) gate14(WriteEn[14],OE[14],RegWrite);
  and #(50) gate15(WriteEn[15],OE[15],RegWrite);
  and #(50) gate16(WriteEn[16],OE[16],RegWrite);
  and #(50) gate17(WriteEn[17],OE[17],RegWrite);
  and #(50) gate18(WriteEn[18],OE[18],RegWrite);
  and #(50) gate19(WriteEn[19],OE[19],RegWrite);
  and #(50) gate20(WriteEn[20],OE[20],RegWrite);
  and #(50) gate21(WriteEn[21],OE[21],RegWrite);
  and #(50) gate22(WriteEn[22],OE[22],RegWrite);
  and #(50) gate23(WriteEn[23],OE[23],RegWrite);
  and #(50) gate24(WriteEn[24],OE[24],RegWrite);
```

```
 and #(50) gate25(WriteEn[25],OE[25],RegWrite);
 and #(50) gate26(WriteEn[26],OE[26],RegWrite);
 and #(50) gate27(WriteEn[27],OE[27],RegWrite);
 and #(50) gate28(WriteEn[28],OE[28],RegWrite);
 and #(50) gate29(WriteEn[29],OE[29],RegWrite);
 and #(50) gate30(WriteEn[30],OE[30],RegWrite);
 and #(50) gate31(WriteEn[31],OE[31],RegWrite);
endmodule
module andmore(g,a,b,c,d,e);
 output g;
 input a,b,c,d,e;
 and #(50) and1(f1,a,b,c,d),
      and2(g,f1,e);
endmodule
module dec5to32(Out,Adr);
input [4:0] Adr; // Adr=Address of register
output [31:0] Out;
not #(50) Inv4(Nota, Adr[4]);
not #(50) Inv3(Notb, Adr[3]);
not #(50) Inv2(Notc, Adr[2]);
not #(50) Inv1(Notd, Adr[1]);
not #(50) Inv0(Note, Adr[0]);
andmore a0(Out[0], Nota,Notb,Notc,Notd,Note); // 00000
andmore a1(Out[1], Nota,Notb,Notc,Notd,Adr[0]); // 00001
andmore a2(Out[2], Nota,Notb,Notc,Adr[1],Note); //00010
andmore a3(Out[3], Nota,Notb,Notc,Adr[1],Adr[0]);
andmore a4(Out[4], Nota,Notb,Adr[2],Notd,Note);
andmore a5(Out[5], Nota,Notb,Adr[2],Notd,Adr[0]);
andmore a6(Out[6], Nota,Notb,Adr[2],Adr[1],Note);
andmore a7(Out[7], Nota,Notb,Adr[2],Adr[1],Adr[0]);
andmore a8(Out[8],  Nota,Adr[3],Notc,Notd,Note);
andmore a9(Out[9],  Nota,Adr[3],Notc,Notd,Adr[0]);
andmore a10(Out[10], Nota,Adr[3],Notc,Adr[1],Note);
andmore a11(Out[11], Nota,Adr[3],Notc,Adr[1],Adr[0]);
andmore a12(Out[12], Nota,Adr[3],Adr[2],Notd,Note);
andmore a13(Out[13], Nota,Adr[3],Adr[2],Notd,Adr[0]);
andmore a14(Out[14], Nota,Adr[3],Adr[2],Adr[1],Note);
andmore a15(Out[15], Nota,Adr[3],Adr[2],Adr[1],Adr[0]);
andmore a16(Out[16], Adr[4],Notb,Notc,Notd,Note);
andmore a17(Out[17], Adr[4],Notb,Notc,Notd,Adr[0]);
andmore a18(Out[18], Adr[4],Notb,Notc,Adr[1],Note);
andmore a19(Out[19], Adr[4],Notb,Notc,Adr[1],Adr[0]);
andmore a20(Out[20], Adr[4],Notb,Adr[2],Notd,Note);
andmore a21(Out[21], Adr[4],Notb,Adr[2],Notd,Adr[0]);
andmore a22(Out[22], Adr[4],Notb,Adr[2],Adr[1],Note);
andmore a23(Out[23], Adr[4],Notb,Adr[2],Adr[1],Adr[0]);
andmore a24(Out[24], Adr[4],Adr[3],Notc,Notd,Note);
andmore a25(Out[25], Adr[4],Adr[3],Notc,Notd,Adr[0]);
andmore a26(Out[26], Adr[4],Adr[3],Notc,Adr[1],Note);
andmore a27(Out[27], Adr[4],Adr[3],Notc,Adr[1],Adr[0]);
andmore a28(Out[28], Adr[4],Adr[3],Adr[2],Notd,Note);
andmore a29(Out[29], Adr[4],Adr[3],Adr[2],Notd,Adr[0]);
andmore a30(Out[30], Adr[4],Adr[3],Adr[2],Adr[1],Note);
andmore a31(Out[31], Adr[4],Adr[3],Adr[2],Adr[1],Adr[0]); // 11111
endmodule
```

Verilog code for a 16-bit register:

```
 //fpga4student.com: FPga projects, Verilog projects, VHDL projects
 // Verilog code for register
module PC_Reg(PCOut,PCin,reset,clk);
 output [31:0] PCOut;
 input [31:0] PCin;
 input reset,clk;
```

```
D_FF dff0(PCOut[0],PCin[0],reset,clk);
D_FF dff1(PCOut[1],PCin[1],reset,clk);
D_FF dff2(PCOut[2],PCin[2],reset,clk);
D_FF dff3(PCOut[3],PCin[3],reset,clk);
D_FF dff4(PCOut[4],PCin[4],reset,clk);
D_FF dff5(PCOut[5],PCin[5],reset,clk);
D_FF dff6(PCOut[6],PCin[6],reset,clk);
D_FF dff7(PCOut[7],PCin[7],reset,clk);
D_FF dff8(PCOut[8],PCin[8],reset,clk);
D_FF dff9(PCOut[9],PCin[9],reset,clk);
D_FF dff10(PCOut[10],PCin[10],reset,clk);
D_FF dff11(PCOut[11],PCin[11],reset,clk);
D_FF dff12(PCOut[12],PCin[12],reset,clk);
D_FF dff13(PCOut[13],PCin[13],reset,clk);
D_FF dff14(PCOut[14],PCin[14],reset,clk);
D_FF dff15(PCOut[15],PCin[15],reset,clk);
D_FF dff16(PCOut[16],PCin[16],reset,clk);
D_FF dff17(PCOut[17],PCin[17],reset,clk);
D_FF dff18(PCOut[18],PCin[18],reset,clk);
D_FF dff19(PCOut[19],PCin[19],reset,clk);
D_FF dff20(PCOut[20],PCin[20],reset,clk);
D_FF dff21(PCOut[21],PCin[21],reset,clk);
D_FF dff22(PCOut[22],PCin[22],reset,clk);
D_FF dff23(PCOut[23],PCin[23],reset,clk);
D_FF dff24(PCOut[24],PCin[24],reset,clk);
D_FF dff25(PCOut[25],PCin[25],reset,clk);
D_FF dff26(PCOut[26],PCin[26],reset,clk);
D_FF dff27(PCOut[27],PCin[27],reset,clk);
D_FF dff28(PCOut[28],PCin[28],reset,clk);
D_FF dff29(PCOut[29],PCin[29],reset,clk);
D_FF dff30(PCOut[30],PCin[30],reset,clk);
D_FF dff31(PCOut[31],PCin[31],reset,clk);
endmodule
```

Recommended Verilog projects:

1. **What is an FPGA? How Verilog works on FPGA**
2. Verilog code for FIFO memory
3. Verilog code for 16-bit single-cycle MIPS processor
4. Programmable Digital Delay Timer in Verilog HDL
5. Verilog code for basic logic components in digital circuits
6. Verilog code for 32-bit Unsigned Divider
7. Verilog code for Fixed-Point Matrix Multiplication
8. Plate License Recognition in Verilog HDL
9. Verilog code for Carry-Look-Ahead Multiplier
10. Verilog code for a Microcontroller
11. Verilog code for 4x4 Multiplier
12. Verilog code for Car Parking System
13. Image processing on FPGA using Verilog HDL
14. How to load a text file into FPGA using Verilog HDL
15. Verilog code for Traffic Light Controller
16. Verilog code for Alarm Clock on FPGA
17. Verilog code for comparator design
18. Verilog code for D Flip Flop
19. Verilog code for Full Adder
20. Verilog code for counter with testbench
21. Verilog code for 16-bit RISC Processor
22. Verilog code for button debouncing on FPGA
23. How to write Verilog Testbench for bidirectional/ inout ports
24. Tic Tac Toe Game in Verilog and LogiSim
25. 32-bit 5-stage Pipelined MIPS Processor in Verilog (Part-1)

26. 32-bit 5-stage Pipelined MIPS Processor in Verilog (Part-2)
27. 32-bit 5-stage Pipelined MIPS Processor in Verilog (Part-3)
28. Verilog code for Decoder
29. Verilog code for Multiplexers
30. N-bit Adder Design in Verilog
31. Verilog vs VHDL: Explain by Examples
32. Verilog code for Clock divider on FPGA
33. **How to generate a clock enable signal in Verilog**
34. **Verilog code for PWM Generator**
35. **Verilog coding vs Software Programming**

DMCA PROTECTED

Privacy Policy | Disclaimer | Sitemap | Contact | Advertise Here
Copyright © 2016-2020 FPGA4student.com All Rights Reserved.